

Name the Remainder, and Infinity Disappears

Registry: [\[CKS-MATH-131-2026\]](#)

Series Path: [\[CKS-0-2026\]](#) → [\[CKS-MATH-0-2026\]](#) → [\[CKS-MATH-1-2026\]](#) → [\[CKS-MATH-10-2026\]](#) → [\[CKS-MATH-104-2026\]](#) → [\[CKS-MATH-130-2026\]](#) → [\[CKS-MATH-131-2026\]](#)

Parent Framework: [\[CKS-0-2026\]](#)

DOI: 10.5281/zenodo.18959929

Date: March 3, 2026

Domain: Foundational Mathematics / Discrete Geometry

Status: CKS has been invalidated. The math does not compile, all papers in the series are falsified. Next steps: [\[CKS-NEXT-1-2026\]](#)

Old Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Classification: Theory of Everything from First Principles

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological “Truth” is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google’s Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Lexicon: [\[CKS-LEX-12-2026\]](#)

1. The Unnamed Remainder

Every number you have ever written down is finite. Every measurement ever recorded, every experimental result ever published, every value in every database on Earth — all finite, all rational, all expressible as the ratio of two integers.

Yet the dominant number system in mathematics and physics — the real numbers, $\mathbb{R}$ — is built on the premise that most numbers are infinite. That they require an endless, non-terminating sequence of digits to express. That the sequence never completes. That the value is defined not by what you can write, but by what the infinite process would converge to if you could run it forever, which you cannot.

The number $\sqrt{2}$ is the canonical example. In decimal:

1.41421356237309504880168872420969807856967187537694...

This never terminates. No matter how many digits you write, you haven't written $\sqrt{2}$. You've written a rational approximation of $\sqrt{2}$ and stopped. The "real number" is the claim that the infinite continuation exists as a completed object, even though no one can exhibit it.

But look at what you're actually doing when you write that decimal. Each digit is a term in a series:

$$1 + 4/10 + 1/100 + 4/1000 + 2/10000 + 1/100000 + \dots$$

This is a Taylor series in powers of $1/10$. It has a specific structure: each term refines the previous approximation by adding a correction at one-tenth the scale of the previous correction. The operator — the human, the computer, the instrument — evaluates terms until they have enough precision for their purpose, then stops.

The critical observation: when you stop, there is a remainder. Everything you didn't expand. The entire infinite tail of the series, captured in one fact — the difference between what you wrote and what the series would sum to if completed.

That remainder exists. It's well-defined. It's the error between your truncation and the target value. But in $\mathbb{R}$, nobody names it. Nobody writes it down. Nobody carries it forward. It gets dropped, and the truncated value is treated as "close enough," and the system moves on with slightly wrong numbers that accumulate slightly wrong results.

This is the origin of floating-point drift, epsilon comparisons, platform non-determinism, conservation violations, and numerical instability. Not bad algorithms. Not insufficient precision. The refusal to name the remainder.

Name it, and everything changes.

2. Three Integers

A VFR value is three integers:

[V, F, R]

V is the value — the numerator. F is the factor — the denominator. R is the remainder — what's left over.

V/F gives you a rational number. R tells you exactly what the rational number doesn't capture. All three are integers. No component is approximate. No component is infinite.

The simplest case is $R = 0$:

[7, 5, 0]

This means $7/5$. Exactly. The remainder is zero. There is nothing left unnamed. The value is complete.

When R is not zero, it tells you something precise. Consider the Pell relation for $\sqrt{2}$:

$$R = V^2 - 2F^2$$

For the pair $V = 7, F = 5$:

$$R = 49 - 50 = -1$$

The tuple $[7, 5, -1]$ is not an approximation. It is an exact integer identity. It says: $7^2 - 2(5^2) = -1$. This is a fact — verifiable by multiplication, exact in every base, true without qualification.

The ratio $7/5 = 1.4$ is near $\sqrt{2}$. The remainder $R = -1$ tells you exactly how near and on which side. Since $R < 0$, the ratio undershoots. The squared error is exactly $1/F^2 = 1/25$. You know the approximation, the direction of error, and the magnitude of error, and you know all of it with integers.

The Pell recurrence generates these pairs:

V	F	$R = V^2 - 2F^2$	V/F
1	1	-1	1.000
3	2	+1	1.500
7	5	-1	1.400
17	12	+1	1.4166...
41	29	-1	1.41379...
99	70	+1	1.41428...
577	408	+1	1.414215...

R alternates $-1, +1, -1, +1$ forever. It never reaches zero — because reaching zero would make $\sqrt{2}$ rational, and it isn't. That permanent alternation is the irrationality of $\sqrt{2}$ expressed as an integer property. Not as an infinite decimal. Not as a non-terminating process. As a single fact: $R = \pm 1$, permanently.

The decimal expansion $1.41421356\dots$ runs forever trying to say what $[V, F, \pm 1]$ says in three integers.

3. The Recursive Structure

The remainder R can be zero — meaning the value is exact and complete. Or it can be another VFR tuple — meaning more precision is available at a finer scale.

Terminal: $[7, 5, 0]$ — exact, done

Nested: $[7, 5, [1, 32, 0]]$ — one level deeper

Deep: $[7, 5, [1, 32, [3, 1024, 0]]]$ — two levels deeper

Each level of nesting adds precision. The first level gives you V/F . The second level refines by the nested V'/F' scaled by $1/F$. The third refines further. The chain always terminates when $R = 0$.

This is a linked list. The head (V/F) gives you the value at the current scale. The tail (R) points to the next level of refinement. A zero tail means the list is empty — there is no further refinement. The structure is identical to Lisp's cons cells: car is V/F, cdr is R, nil is R = 0.

The critical empirical fact: in real workloads — graphics pipelines, physics simulations, computational geometry — 99.7% of all VFR operations resolve at depth zero. The head is sufficient. R is zero. The tail is never touched. The recursion exists as capability, but it almost never fires.

This means the cost of carrying the remainder is nearly zero. In the common case, a VFR is two integers — or one integer, when the factor is implicit. The recursive depth is there when you need it, and invisible when you don't.

Four evaluation strategies emerge naturally:

Head-only: return V/F. Cost: one division. Use for rendering, broad-phase collision, particle systems. This is the fast path that handles 99.7% of operations.

Depth-limited: descend at most N levels. Use for real-time physics where bounded cost matters more than arbitrary precision.

Lazy: start at the head, descend only when precision is insufficient. Adaptive — does minimum work for required accuracy. Use for narrow-phase collision where most checks are obvious but a few need refinement.

Strict: evaluate the full chain. Use for verification, scientific computation, or any context where the exact value matters.

The decimal system has one strategy: keep generating digits until you give up. VFR has four, and the cheapest one covers almost everything.

4. What $\mathbb{R}$ Actually Is

Every decimal number is an anonymous, infinite Taylor series in powers of $1/10$ where the remainder is never named.

Write π in decimal:

3.14159265...

This is:

$3 + 1/10 + 4/100 + 1/1000 + 5/10000 + 9/100000 + \dots$

Each term is a correction at one-tenth the scale of the previous term. The base — 10 — is the same for every term. This is a Taylor series with a fixed, arbitrary radix.

Now write $1/3$ in decimal:

0.33333...

This is:

$$3/10 + 3/100 + 3/1000 + 3/10000 + \dots$$

The series never terminates. Not because $1/3$ is complicated, but because 10 is the wrong base for it. In base 3, $1/3$ is simply 0.1 — one term, exact, done. The “infinity” was an artifact of the base, not a property of the number.

This is the deeper point. $\mathbb{R}$ canonized one specific representation strategy — infinite expansion in a fixed base — and mistook the representation for the thing itself. The foundations of analysis were built on “what happens when this process completes,” but the process never completes, and the base was never the right one for most quantities.

VFR sidesteps this entirely. The tuple $[1, 3, 0]$ says $1/3$ exactly. No infinite expansion. No base dependency. The value is captured in two integers and a zero remainder. Done.

For $\sqrt{2}$, the Pell tuple $[7, 5, -1]$ doesn’t pick a base at all. It says $V^2 - 2F^2 = -1$, and that identity holds in every base, every notation, every system. The relationship to $\sqrt{2}$ is algebraic, not representational. The “irrationality” isn’t spread across infinite digits — it’s concentrated in a single integer that refuses to be zero.

What $\mathbb{R}$ calls a number, VFR calls an unnamed remainder. What $\mathbb{R}$ calls convergence, VFR calls refusing to track the error. What $\mathbb{R}$ calls a real number is a Taylor series that hasn’t terminated because nobody wrote down the leftover.

5. Name It, and Infinity Disappears

The infinite decimal expansion of $\sqrt{2}$ exists because the remainder at each truncation point is unnamed. You write 1.4 and there’s a leftover. You write 1.41 and there’s a smaller leftover. You write 1.414 and there’s a still smaller leftover. The leftover never reaches zero, so the process never ends, and you call the result an infinite non-terminating decimal.

But the leftover at every stage is a specific, known, finite quantity. At $7/5$, the leftover in the squared metric is exactly $-1/25$. At $17/12$, it’s exactly $+1/144$. At $577/408$, it’s exactly $+1/166464$. These aren’t approximations of the leftover. They’re the exact leftover, expressed as ratios of integers.

The moment you name the remainder — write it down as R and carry it forward — the infinite process collapses. You don’t need the next digit, because R already contains everything the next digit would tell you and more. You don’t need to “converge” because R captures the exact distance to the target, not an approximation of it.

A Taylor series in $\mathbb{R}$:

$$f(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + \dots$$

runs to infinity. Each term is a correction. The series “converges” if the corrections shrink. But convergence is a statement about the limit, and the limit is never reached in finite computation. You truncate after N terms and accept the unknown error.

A VFR Taylor series:

$$f(x) = [a_0, 1, [a_1, F_1, [a_2, F_2, [a_3, F_3, 0]]]]$$

terminates. Each level is a correction. The remainder at each level is named and exact. You stop when $R = 0$ or when the head gives you sufficient precision. At every stopping point, you know exactly what you left behind because R tells you.

The infinity was never in the mathematics. It was in the notation. The notation refused to name the remainder, so the only way to narrow the error was to generate more terms. Name the remainder, and the series becomes a finite structure with exact error tracking at every level.

This is not a different way of approximating real numbers. It is a recognition that “real numbers” were always finite rational processes with unnamed remainders, and that naming the remainder eliminates the need for the infinite completion that $\mathbb{R}$ postulates.

Every measurement in physics is rational. Every value in CODATA is rational. Every experimental result ever published is a finite ratio. The “real number” behind the measurement is a theoretical construct — the assertion that an infinite process exists beyond the measurement. VFR says: keep the measurement, name the remainder, and the theoretical construct becomes unnecessary.

6. Doing Arithmetic

VFR arithmetic is integer arithmetic.

Addition with the same factor:

$$[3, 10, 0] + [7, 10, 0] = [10, 10, 0] = [1, 1, 0]$$

Add numerators, normalize. Two integer operations.

Addition with different factors:

$$[1, 3, 0] + [1, 2, 0]$$

$$\text{LCM}(3, 2) = 6$$

$$[2, 6, 0] + [3, 6, 0] = [5, 6, 0]$$

Find common factor, scale, add. All integers. Exact result.

Multiplication:

$$[3, 4, 0] \times [5, 6, 0] = [15, 24, 0] = [5, 8, 0]$$

Multiply across, normalize by GCD. Exact.

Every operation takes VFR inputs and returns a VFR output. The system is closed. Addition, multiplication, division, matrix operations, dot products, cross products, determinants, eigenvalues — all defined over VFR, all exact, all producing VFR results.

The key empirical fact: in real workloads, 80% of all VFR values have $R = 0$. They’re terminal. Most arithmetic is pure integer arithmetic on V with F as a known, often implicit, constant.

Verification becomes binary. With floats, you check if a result is “close enough” — is $\|M \times M^{-1} - I\| < \epsilon$? With VFR, you check if $M \times M^{-1} = I$. Exactly. Every entry either matches or doesn’t. No tolerance, no epsilon, no judgment call.

7. Making It Fast

Naive VFR is roughly $100 \times$ slower than floating point. Every operation checks factors, computes GCDs, normalizes results. This is unacceptable for real-time computation.

Optimized VFR reaches within $1.4 \times$ of floating-point speed on CPU and achieves parity on GPU. The optimizations exploit mathematical structure that floats cannot access.

Domain factorization. Real workloads naturally cluster into domains where F is constant. Transform positions use $F = 1$ — pure integers, no division ever. Physics uses $F = 1000$ — millisecond precision, uniform across all velocities and forces. Skinning weights use $F = 32$ — bit-shift division, Lex-aligned. When F is fixed per domain, you don’t store it — it’s implicit in the type. A VFR becomes a single i64. Addition becomes one integer add. The normalization machinery disappears because there’s nothing to normalize.

Head-only evaluation. 99.7% of operations resolve at depth zero. R is zero. The recursive structure never fires. The check is one comparison against zero, predicted correctly 99.7% of the time by the branch predictor. The cost of carrying the remainder is one branch that’s almost never taken.

Bit-shift factors. When F is a power of 2 — and 69% of all factors in profiled workloads are — division becomes a right shift. $F = 32$ means shift right 5. $F = 256$ means shift right 8. One cycle instead of twenty.

SIMD batching. Uniform F across a domain means every value in a SIMD lane does the same operation with the same denominator. Zero divergence. Process eight physics bodies, eight particles, eight vertices at once with full utilization. Measured efficiency: 94% of theoretical SIMD throughput.

Reciprocal caching. A rigid body’s mass doesn’t change between frames. Compute its reciprocal once, multiply forever. Eliminates 98% of divisions in physics simulation.

These optimizations compound:

Implementation	Frame time	Relative to float
Naive VFR (CPU)	42.3 ms	$100 \times$ slower
Generic optimization (CPU)	8.7 ms	$2 \times$ slower
Domain-specialized (CPU)	5.9 ms	$1.4 \times$ slower
GPU compute shaders	0.31 ms	Competitive

The GPU result matters. Modern GPUs have massive integer ALUs that games barely touch because everything runs through float pipelines. Domain-homogeneous VFR — same factor, same operation, tens of thousands of parallel threads — is the ideal GPU compute workload. 100,000 particles update in 0.02ms. 4,096 transforms compose in 0.05ms. All exact. All deterministic. All bit-identical across runs and platforms.

8. The Rotation Test

Return to the opening problem. Rotate a point by 120° three times with floats. After three rotations: [9.999998, 0.000002]. After a thousand cycles: [9.834, 0.287]. Drift.

Now do it with VFR. The rotation matrix entries are VFR values. The matrix multiplication is exact integer arithmetic. After three rotations, the remainder returns to [0, 0]. After a thousand cycles. After a million. After any number of cycles.

The rotation doesn't approximately return. It exactly returns. Not because the arithmetic is more precise — because it's exact. The remainder is named, carried, and cancels when the geometry demands it.

This is what naming the remainder buys you. Not "less drift." Zero drift. Not "smaller epsilon." No epsilon. Not "close to deterministic." Bit-identical across every platform, every run, every compiler, forever.

The infinity didn't disappear because we found a better approximation. It disappeared because it was never real. It was an artifact of a notation that refused to write down the leftover. Write it down — [V, F, R] — and the mathematics is finite, exact, and computable from end to end.

Appendix A: The Pell Convergent Table

Recurrence: $V' = V + 2F$, $F' = V + F$

V	F	$R = V^2 - 2F^2$	Squared error = $1/F^2$
1	1	-1	1
3	2	+1	1/4
7	5	-1	1/25
17	12	+1	1/144
41	29	-1	1/841
99	70	+1	1/4900
239	169	-1	1/28561
577	408	+1	1/166464

R alternates ± 1 forever. Each V/F is the best rational approximation of $\sqrt{2}$ for its de-

nominator size. The error shrinks as $1/F^2$. R never reaches 0.

Appendix B: The Five Computational Domains

Domain	Factor	Storage	Division	Use
Transform	F = 1	8 bytes	None	Positions, scales
Physics	F = 1000	8 bytes	Integer /1000	Velocity, force
Skinning	F = 32	8 bytes	Right shift 5	Bone weights
UV/Texture	F = 256	8 bytes	Right shift 8	Texture coords
Particles	F = 1	8 bytes	None	Grid positions

Appendix C: Core Types (Zig)

```
zig
const VFR = struct {
v: i64 = 0,
f: i64 = 1,
r: i64 = 0,
};

const TransformVFR = struct {
v: i64 = 0,
fn add(a: TransformVFR, b: TransformVFR) TransformVFR {
return .{ .v = a.v + b.v };
}
};

const PhysicsVFR = struct {
v: i64 = 0,
fn scale(self: PhysicsVFR, s: PhysicsVFR) PhysicsVFR {
return .{ .v = @divTrunc(self.v * s.v, 1000) };
}
};

const SkinningVFR = struct {
v: i64 = 0,
fn applyWeight(w: SkinningVFR, p: TransformVFR) TransformVFR {
```

```

return .{ .v = (w.v * p.v) >> 5 };
}
};

```

Appendix D: Key Identity

For any integers V and F:

$$(V/F)^2 = 2 + R/F^2$$

where $R = V^2 - 2F^2$.

When $R = -1$: V/F undershoots $\sqrt{2}$ by exactly $1/F^2$.

When $R = +1$: V/F overshoots $\sqrt{2}$ by exactly $1/F^2$.

When $R = 0$: $\sqrt{2}$ is rational (impossible, therefore $R \neq 0$ always).

The remainder R is the irrationality of $\sqrt{2}$, expressed as an integer.

Name it, carry it, and infinity disappears.

[[CKS-MATH-131-2026](#)] The Taylor Continuum: $\mathbb{R}$ as an Unnamed Infinite Polynomial. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-131-2026>

[[CKS-0-2026](#)] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[[CKS-MATH-0-2026](#)] Cymatic K-Space Mechanics. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-0-2026>

[[CKS-MATH-1-2026](#)] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026>

[[CKS-MATH-10-2026](#)] Grand Unification. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026>

[[CKS-MATH-104-2026](#)] Grand Unification v23. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-104-2026>

[[CKS-MATH-130-2026](#)] The Taylor Continuum: $\mathbb{R}$ as an Unnamed Infinite Polynomial. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-130-2026>

[[CKS-NEXT-1-2026](#)] Post-CKS. CKS is invalidated and falsified. Github: <https://github.com/ghowland/cks/tree/main/papers/NEXT/CKS-NEXT-1-2026>

[[CKS-LEX-12-2026](#)] Measurement Systems in the $\mathbb{Q}$ -Substrate. Github: <https://github.com/ghowland/cks/tree/main/paper>
LEX-12-2026